

CPCS 405 – 2nd Semester 2026 – Final Project

Due: Thursday, May 21, 2026 @ 11:59 pm

About the Project – Online Exam/Quiz Application

In the final project, you are tasked to develop a simple **Online Exam/Quiz** application. This project combines the concepts you learned in this class, including JavaFX, Networking, Multithreading, I/O, Collections, and Database programming, to deliver a functional, efficient, and user-friendly application. Below is a detailed breakdown of the requirements that need to be implemented.

Deadline: Thursday, May 21, at 11:59 pm.

Grade: 20% of the final grade.

This is a group project. [Please contact your instructor to form your group by April 23.](#)

Learning Outcomes

In this project, you will apply concepts related to JavaFX, multithreading, File I/O, collections, socket programming, Java JDBC, and Internationalization.

Server-Side Description and Screenshots

Step 1 – Creating a text file for the exam questions.

In order to create a new exam, the exam questions and answers must be imported from a text file. Therefore, a *text file* must be created for the exam. **Each line** in the text file represents a single question in the exam. The content of the line depends on the type of question as follows:

- **Question Type:** which can be either *TF* for True/False questions, or *MCQ* for multiple choice questions.
- A required **semicolon**.
- **Question Text:** the actual text of the question.
- A required **semicolon**.
- In case the question is of type *TF*:
 - The **correct answer** must be provided next, which can be either *TRUE* or *FALSE*.
- In case the question is of type *MCQ*, four choices must be given in the line as follows:
 - The text of the **first choice** is given + a **required semicolon** + either *CORRECT* or *INCORRECT* depending on whether this is the correct choice + a **required semicolon**.
 - The text of the **second choice** is given + a **required semicolon** + either *CORRECT* or *INCORRECT* depending on whether this is the correct choice + a **required semicolon**.
 - The text of the **third choice** is given + a **required semicolon** + either *CORRECT* or *INCORRECT* depending on whether this is the correct choice + a **required semicolon**.
 - The text of the **fourth choice** is given + a **required semicolon** + either *CORRECT* or *INCORRECT* depending on whether this is the correct choice.

CPCS 405 – 2nd Semester 2026 – Final Project

Due: Thursday, May 21, 2026 @ 11:59 pm

The following is a sample of a valid file for an exam/quiz containing two questions:

```
TF;Java is an object-oriented programming language;TRUE
MCQ;Single-line comments in Java start with;# ;INCORRECT; //; CORRECT; !; INCORRECT; $; INCORRECT
```

Notes:

1. In this project, each MCQ has one correct answer (i.e., there is no need to consider multiple correct answers for an MCQ). The application must validate the file during file import.
2. Each question is allocated 1 point. Therefore, the total points of the exam are equal to the number of questions.

Step 2.A – Initiating the server for online exams.

When the application starts at the server, the following JavaFX window is shown. The user (the exam admin in this case) clicks “Start Exam Server”.

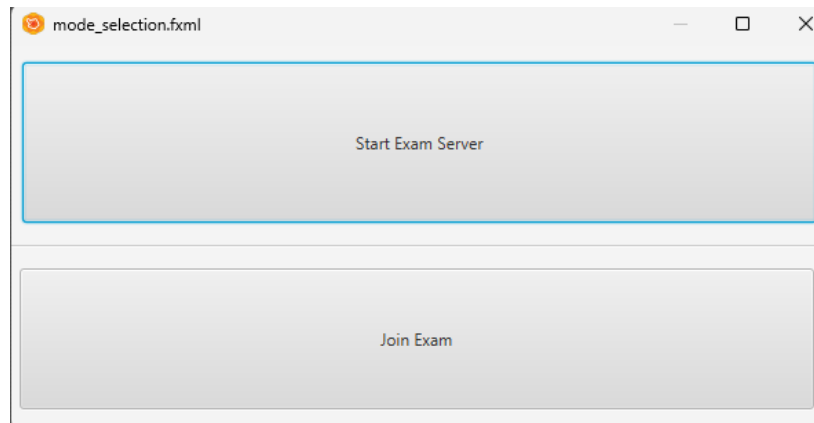


Figure 1

Note: In a realistic setting, the user must be authenticated/authorized, and security measures (e.g., encryption) must be applied. Given that these topics are outside the course’s scope, and to limit the scope of the project, there is no need to consider security aspects.

The application then displays the following JavaFX screen:

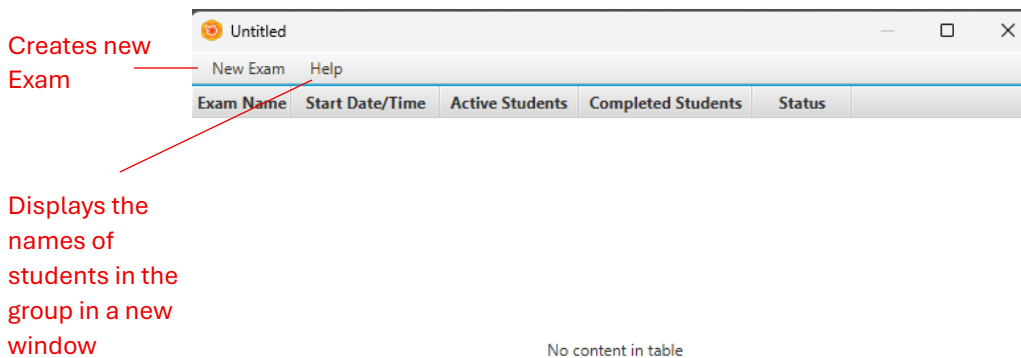


Figure 2

CPCS 405 – 2nd Semester 2026 – Final Project

Due: Thursday, May 21, 2026 @ 11:59 pm

This window should show, using a TableView, the information of created exams as follows:

- Column 1 (Exam Name): the name of the exam + semester + year.
- Column 2 (Start Date/Time): the date and time at which this exam has started.
- Column 3 (Active students): the number of students who are currently taking the exam but have not finished yet.
- Column 5 (Completed students): the number of students who have completed this exam.
- Column 6 (Status): the status of the exam, which can be either “Active” or “Ended”. The exam is considered active once it has been created by the admin user. The exam is considered “Ended” after 60 minutes from its starting time.

Initially, the TableView will not have any exam data to be displayed.

The user (Exam Admin) can press on “New Exam” from the main menu to create a new exam. The application should display the following JavaFX screen:

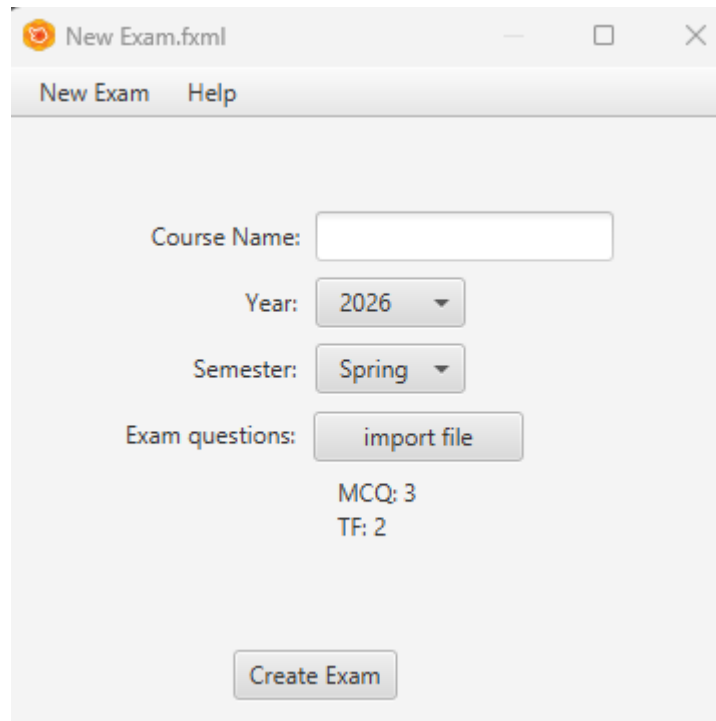


Figure 3

The user then enters the exam name, e.g., “CPCS405”, selects the year from a ChoiceBox, selects the semester from ChoiceBox, presses the “Import File” button to import the file from step 1, and then presses the “Create Exam” button to create the exam. The year ChoiceBox must include the current year (e.g., “2026”) as well as two previous years. For the semester ChoiceBox, the values “Spring”, “Fall”, and “Summer” must be included. When the user presses “Import File”, a window that allows the user to select a file (from step 1) must be displayed using [FileChooser](#). The application must open and read the file and summarize the number of MCQ and TF questions. The application should display an error message if the file does not follow the correct format.

CPCS 405 – 2nd Semester 2026 – Final Project

Due: Thursday, May 21, 2026 @ 11:59 pm

The application should then create appropriate instances from the following classes according to the data entered for this exam:

- Exam class: create one instance from this class per exam, according to the data entered by the user. The startDateTime can be set to be the current date/time using `LocalDateTime.now()`
- Question classes: for each question, create an instance from the appropriate question class, and add this instance to the questions collection.

Important note: you are free to enhance these classes as you need (for example, adding more fields, adding constructors, getters/setters, changing the modifiers, and adding more supporting classes). However, your code must contain these classes and the provided attributes shown below.

```
class Exam {
    string courseName;
    string year;
    string semester;
    LocalDateTime startDateTime; //the date and time at which this exam has started
    Collection<Question> questions; //construct an appropriate collection type to be used
}
class Question implements Serializable {
    string text; //the question text
}
class QuestionTF extends Question {
    boolean answer; //question answer
}
class QuestionMCQ extends Question {
    string choice1;
    boolean choice1Correct;
    string choice2;
    boolean choice2Correct;
    string choice3;
    boolean choice3Correct;
    string choice4;
    boolean choice4Correct;
}
```

The application must maintain an appropriate collection of all exam instances.

Step 2.B – Initiating the server for online exams.

In addition to step 2.A, when the user clicks “Start Exam Server”, the application must create a new thread responsible for handling messages from students using Java Sockets (TCP/IP). This thread must in a loop:

- Listen for incoming client connections (i.e., students) using port 3000.
- Spawn a new thread for handling the new connection.

The spawned thread for each connection must use sockets as follows:

CPCS 405 – 2nd Semester 2026 – Final Project

Due: Thursday, May 21, 2026 @ 11:59 pm

- The server receives as a string the student's ID. The server must store this information in an appropriate collection. The thread then sends back to the client a list of the available exams (i.e., course names). The server waits for the next message from the client.
- The client picks one of the exams (see Client-Side details below) and sends a message to the server containing the desired exam.
- The server enters a loop, in which it sends the exam questions of the selected exam, one by one, to the client as follows:
 - The server sends the next exam question to the client. **Note: the server should not send any answers to the client.** In addition, the server should send only one object to the client containing the TF/MCQ details. Sending multiple objects in different messages (e.g., choices of MCQ in different messages) is not accepted.
 - The client sends the answer to this question to the server. The server updates the score for this exam for this client/student (1 point for each question).
 - The server repeats the previous two steps until the exam is completed. There is no need to allow the client (student) to go backward to previous questions.
- The server sends a message to the client with the final score. The server also inserts into the database the score of this student.

Note: to enable this communication, you are allowed to send objects of any type that you choose (even create new types). Ensure that no answers are accidentally sent to clients. You can maintain any necessary collections to store and track exam sessions (e.g. students taking exams).

The server should not allow any client/student after the exam is finished (60 minutes after creation). If a student's connection is interrupted during the exam, the student should be allowed to resume the exam from the last question before the interruption, given that the exam has not finished.

=====

Client-Side Description and Screenshots

=====

At the client-side, the same screen shown in Figure 1 is displayed to the user (who represents a student). However, the user presses "Join Exam" in this case. The following JavaFX is shown:

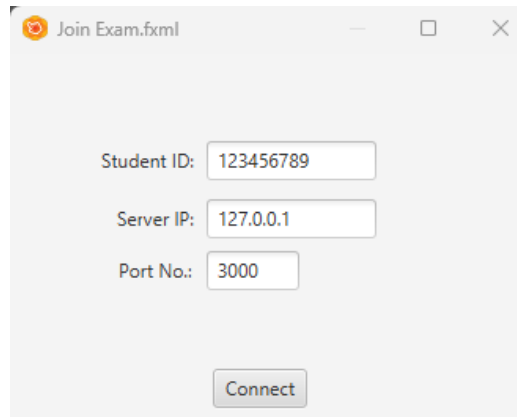


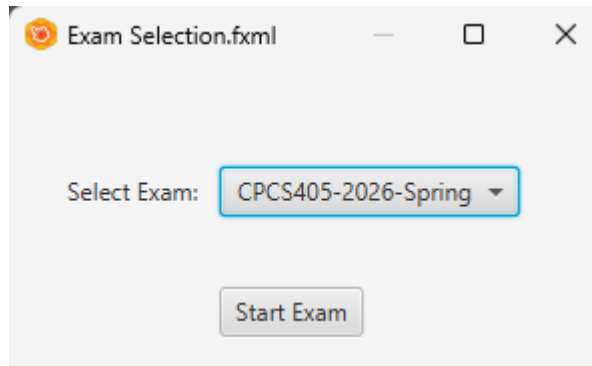
Figure 4

CPCS 405 – 2nd Semester 2026 – Final Project

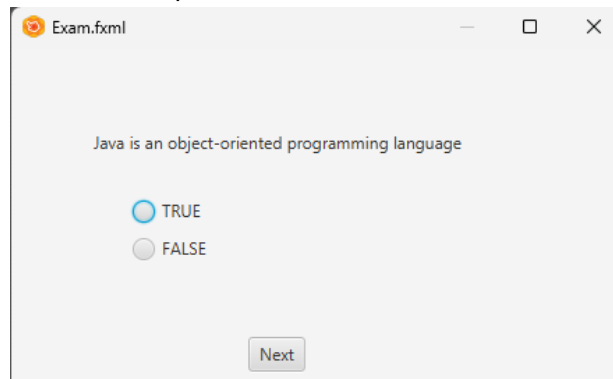
Due: Thursday, May 21, 2026 @ 11:59 pm

The user (i.e., student) enters their student ID, server IP, and port number, then clicks connect. The application must then create a connection to the server using sockets (TCP/IP):

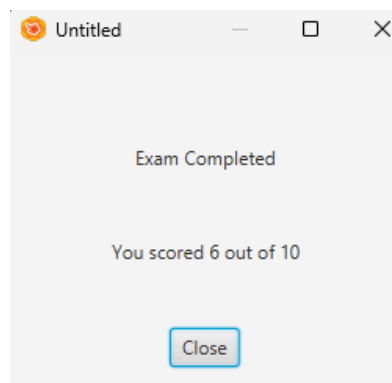
- The client first sends to the server a string containing the student ID.
- The client then receives a list of available exams. The application must then display a ChoiceBox containing the list of exams. The user (student) selects the exam, and click “Start Exam”



- The client sends to the server the selected exam to be taken.
- The client receives from the server the exam questions, one question at a time. For each question, the user answers the question (using RadioButtons) and clicks “Next”. The client sends to the server the selected answer and then receives the next question. This process is continued until the exam is completed.



- After answering all questions, the client receives from the server the final score for the user. The client can now terminate the connection with the server.



CPCS 405 – 2nd Semester 2026 – Final Project

Due: Thursday, May 21, 2026 @ 11:59 pm

Important Considerations:

1. You are allowed to decide the type of objects to be sent between clients and the server. For example, you can send a string to represent the course name in step 1, or you can send a complex serialized object.
2. In both clients and server, you can implement any additional classes you desire. You can also use any collections for storing any necessary information.
3. Make sure to update the screen shown in Figure 2 by showing all exams, start date/time, active students, number of completed students, and the status.
4. Ensure to handle exceptions properly and show error messages using JavaFX.

Database

In this project, you should use a MySQL database to store exam results. The easiest way to install MySQL on your device is by using XAMPP (<https://www.apachefriends.org/>). The database is installed at the server side/node.

After a student completes an exam, the server must store the results in a table named ExamResults. The table consists of the following columns:

- ID (int): an autoincrement primary key.
- Student ID (text): the ID of the student.
- Course Name (text): the name of the course
- Year (text): the exam year
- Semester (text): the semester.
- Score (int): the score the student received.

Logging

The server must maintain a text log file for the key events as follows:

1. When an exam is created.
2. When a client connects
3. The exam that the client/student chooses to take
4. The score the client/student received.
5. Any other major events

Save the file in the same project directory.

CPCS 405 – 2nd Semester 2026 – Final Project

Due: Thursday, May 21, 2026 @ 11:59 pm

Internationalization

Using the concepts of Internationalization, display the text of the first screen shown in Figure 1.

Important Notes

1. You have the flexibility to design the class diagram of this application. However, make sure to meet all the above-mentioned requirements.
2. Ensure that any errors/exceptions are handled appropriately. Also, ensure to appropriately display any error messages using JavaFX.
3. If you have any questions regarding the project, don't hesitate to contact the instructor.

Grading Rubric

The following criteria will be used to grade the project:

Server-side Functionality	10 points	The specified requirements are met, including the following: • Use of JavaFX • Importing exams from files and creation • The ability to display exams, their statuses, and the number of students. • Socket communications • Database connection establishment, queries, and table updates • Appropriate error/exception handling • Logging • Internationalization
Client-side Functionality	10 points	The specified requirements are met regarding: • Use of JavaFX • Socket communications • The ability to take exams, take questions one by one, and display the final score. • Appropriate error/exception handling.
Total	20	

- Note that if the program fails to compile OR the program runs but produces substantial runtime errors and exceptions that prevent obtaining the expected results in most cases,

CPCS 405 – 2nd Semester 2026 – Final Project

Due: Thursday, May 21, 2026 @ 11:59 pm

then a 0% grade will be received. If the program cannot be properly tested (e.g., due to lack of GUI interfaces), then 0% grade will be received.

- Missing the deadline will result in a 15% deduction for each day beyond the deadline.
- Plagiarism will result in receiving zero points for this project.

What to submit?

- Submit your complete Java project.
- Submit at least one sample text file for an exam that you used to test your application
- Submit a script containing a Create Table statement to create the DB Table. You can run the following command in MySQL to generate this statement automatically:
`SHOW CREATE TABLE <name of table>;`
- **Every student in the group must submit the above files for grading.**